

MEMORIA DEL PROYECTO

Implantación y Customización de Odoo

– Club Deportivo de Pádel –



Autores:

Rubén Flores García - Àngel Martínez García

ÍNDICE

MEMORIA DEL PROYECTO	1
Implantación y Customización de Odoo	1
-- Club Deportivo de Pádel --	1
1. Escenario y necesidades	2
1.1 Descripción de la organización	3
1.2 Problemas actuales	3
1.3 Cómo la customización de Odoo resuelve los problemas	3
1.4 Beneficios obtenidos	4
2. Documentación técnica	5
2.1 Fuentes consultadas	5
Documentación oficial de Odoo 16	5
Recursos técnicos adicionales	5
2.2 Uso de herramientas de IA.....	7
ChatGPT - Fase inicial del proyecto	7
Claude Code (Anthropic) - Fase de mejoras (Mayo 2026, tercer trimestre)	11
Claude Code - Fase de corrección de errores (Mayo 2026)	17
2.3 Pasos realizados	19
2.4 Códigos y módulos creados	20
2.5 Modelos.....	23
.....	25
2.6 Backup de la base de datos	25
3. Customizaciones realizadas.....	25
3.1 Modificación de campos y vistas.....	25
3.2 Gestión de clientes, proveedores y productos.....	25
3.3 Creación de módulo propio	25
3.4 Cambios en el entorno gráfico	25
3.5 Mejoras extra realizadas.....	25
4. Customizaciones realizadas.....	25
4.1 Entorno de Desarrollo	25

4.2 Arquitectura de Odoo.....	25
5. Diseño del Modelo de Datos	25
5.1 Modelo Principal: Pistas (club_padel.pista)	25
5.2 Modelo de Reservas (club_padel.reserva).....	25
5.3 Modelo de Jugador (herencia res.partner)	25
6. Seguridad y Permisos.....	25
7. Personalización del Entorno Gráfico	25
7.1 Gestión de Assets CSS	25
8. Metodología de Trabajo	25
8.1 División del trabajo entre miembros del equipo.....	25
8.2 Valoración del trabajo - Rubén Flores	25
8.3 Valoración del trabajo - Àngel Martínez.....	25
8.4 Metodología incremental	25
9. Pruebas Realizadas	25
9.1 Pruebas funcionales	25
9.2 Pruebas de integración entre módulos	25
9.3 Pruebas de consistencia y actualización	25
10. Rendimiento y Optimización	25
11. Escalabilidad del Proyecto	25
12. Plan de Mantenimiento	25
13. Valor Técnico del Proyecto	25
14. Conclusión.....	25

1. Escenario y necesidades

1.1 Descripción de la organización

La organización elegida es un club deportivo de pádel de tamaño medio. Actualmente gestiona sus operaciones con hojas de cálculo y herramientas desconectadas, lo que genera ineficiencias, errores y falta de control. El club dispone de:

- 6 pistas de pádel (indoor y outdoor).
- Gestión de socios y clientes ocasionales.
- Venta de productos deportivos: palas, pelotas, camisetas y accesorios.
- Organización de torneos.
- Proveedores de material deportivo.
- Gestión administrativa básica: facturación y control de clientes.

1.2 Problemas actuales

Antes de la implantación, el club presentaba los siguientes problemas operativos:

- Información dispersa en hojas Excel y en papel.
- Sin control centralizado de clientes y proveedores.
- Sin trazabilidad de ventas de productos.
- Gestión manual y propensa a errores en las reservas de pistas.
- Imagen digital poco profesional.
- Dificultad para generar informes y estadísticas de negocio.

Esta situación provocaba pérdida de tiempo, errores administrativos y falta de control financiero.

1.3 Cómo la customización de Odoo resuelve los problemas

Se ha implementado Odoo 16 Community como ERP centralizado mediante el desarrollo de tres módulos propios que trabajan de forma modular e integrada:

- club_padel → Gestión de pistas y reservas del club.
- tienda_padel → Gestión de productos deportivos.
- club_padel_tienda → Integración entre reservas y productos mediante un sistema de carrito por reserva.

Esta arquitectura modular permite separar responsabilidades, mantener el código organizado y facilitar futuras ampliaciones sin afectar al núcleo del ERP.

1.4 Beneficios obtenidos

- Centralización de todos los datos del club en un único sistema.
- Reducción significativa de errores administrativos.
- Mayor control de inventario de productos deportivos.
- Mejora de imagen corporativa gracias a la personalización gráfica.
- Arquitectura escalable y preparada para futuras ampliaciones.
- Base digital profesional para el club.

1.5 Integración real con el ERP nativo

Un aspecto clave del proyecto es que los clientes, proveedores y productos no viven en tablas aisladas: son objetos nativos de Odoo. Los 3 proveedores de material deportivo (Bullpadel, Head, Dunlop) y los 7 socios del club son registros reales de `res.partner`, accesibles desde el módulo Contacts de Odoo con todos sus datos de contacto. Los 8 productos de la tienda son registros de `product.template` con precio de venta, precio de coste y disponibles en el catálogo de productos de Odoo. Esto significa que el sistema está preparado para conectar con los módulos de Ventas o Compras de Odoo en el futuro sin migración de datos.

2. Documentación técnica

2.1 Fuentes consultadas

Documentación oficial de Odoo 16

Toda la base del proyecto se apoya en la documentación oficial de Odoo. A continuación, se detalla cada fuente, su URL y en qué parte concreta del proyecto se utilizó:

Fuente	URL	Para qué se usó
Odoo 16 Developer Documentation	odoo.com/documentation/16.0/developer.html	Estructura de módulos, <code>__manifest__.py</code> , campos, vistas
Odoo ORM API Reference	...reference/backend/orm.html	<code>@api.depends</code> , <code>@api.constrains</code> , <code>@api.onchange</code> , <code>_inherit</code>
Odoo View Architectures	...reference/backend/views.html	Formularios, listas, kanban, calendar, graph, pivot, search
Odoo Security — Access Rights	...reference/backend/security.html	Fichero <code>ir.model.access.csv</code>
Odoo QWeb Reports	...reference/backend/reports.html	Informe PDF de reserva con plantilla QWeb
Odoo Mail — Chatter	...reference/backend/mixins.html	<code>mail.thread</code> , <code>mail.activity.mixin</code> , <code>tracking=True</code>
Odoo Data Files	...reference/backend/data.html	Archivos XML de datos demo (<code>noupdate=1</code>)
Odoo Frontend Assets	...reference/frontend/assets.html	CSS personalizado con <code>web.assets_backend</code>

Recursos técnicos adicionales

Fuente	URL	Para qué se usó
Foro oficial Odoo Community	odoo.com/forum/help-1	Resolución de errores de carga de módulos
Stack Overflow (etiqueta odoo)	stackoverflow.com/questions/tagged/odoo	Problemas con XML views y assets
GitHub — módulos base Odoo	github.com/odoo/odoo/tree/16.0/addons	Referencia de estructura de módulos
Docker Hub — imagen odoo:16.0	hub.docker.com/_/odoo	Configuración del entorno Docker

Cómo se dedujeron las implementaciones a partir de las fuentes

A continuación, se explica el proceso real de consulta y cómo cada lectura llevó a una decisión concreta de implementación:

Estructura del módulo: Leyendo la sección "Module structure" de la documentación oficial, vimos que todo módulo necesita un **manifest.py** con las claves name, version, depends y data. Esto nos llevó a crear las tres carpetas (club_padel, tienda_padel, club_padel_tienda) con esa misma estructura.

Decoradores ORM: En la referencia de la ORM API, la sección "Computed fields" explica `@api.depends` y la sección "Constraints" explica `@api.constrains`. Al leer los ejemplos, dedujimos que `@api.depends` era para recalcular valores automáticos (como el total de la reserva) y `@api.constrains` para validar datos antes de guardar (como evitar precios negativos o solapamientos).

Vistas XML: La sección "View Architectures" documenta los tipos de vista: tree, form, graph, pivot, calendar. Revisando los ejemplos de graph y pivot, vimos que los campos llevan atributos `type="row"`, `type="col"` y `type="measure"`, lo que nos permitió implementar las vistas de análisis de reservas.

ir.model.access.csv: La sección de Security muestra el formato exacto del CSV con las columnas id, name, model_id:id, group_id:id, perm_read, perm_write, perm_create, perm_unlink. Aplicamos este formato para dar permisos diferenciados a administradores y usuarios básicos.

Informes QWeb: La documentación de Reports explica cómo declarar un `ir.actions.report` en XML y crear la plantilla QWeb con `t-if`, `t-foreach` y `t-field`. Esto nos permitió crear el informe PDF de confirmación de reserva.

2.2 Uso de herramientas de IA

A lo largo del proyecto se utilizaron dos herramientas de IA: ChatGPT (OpenAI) en la fase inicial y Claude Code (Anthropic) en la fase de mejoras y corrección de errores. A continuación se documentan las conversaciones reales con el contexto exacto, los prompts utilizados, las respuestas obtenidas y cómo se aplicaron.

ChatGPT - Fase inicial del proyecto

Herramienta: ChatGPT (chat de grupo compartido entre Rubén y Ángel) · Fecha: 23–25 de febrero 2026

Entorno: Ubuntu 22 · Odoo 16 como servicio systemd (odoo16.service) · usuario operador_odoo

Interacción 1: Error 500 al añadir el módulo Tienda

Prompt utilizado:

Hemos creado el apartado de Tienda en nuestro módulo de Odoo, pero al intentar acceder obtenemos un error 500. Adjuntamos el código completo del módulo. ¿Puedes identificar la causa del error y explicarnos qué está fallando exactamente?

Respuesta obtenida:

ChatGPT identificó dos causas del error 500: (1) el archivo `models/reserva_inherit_tienda.py` tenía código duplicado y una línea con error de sintaxis que rompía la carga del módulo; (2) `models/__init__.py` importaba `reserva_tienda_inherit` pero el archivo real se llamaba `reserva_inherit_tienda.py`, provocando un `ImportError`. Además, el menú referenciaba una acción `accion_ventas_tienda` que no estaba definida en ningún XML.

Cómo se aplicó:

Se corrigieron los dos archivos Python y se creó la acción de ventas en el XML. El error 500 desapareció y Odoo arrancó correctamente.

Interacción 2: El menú Tienda no aparece aunque el XML está bien

Prompt utilizado:

Hemos corregido el menú según tus indicaciones, pero el apartado Tienda sigue sin aparecer en la barra de navegación. Hemos reiniciado el servicio y actualizado el módulo pero el resultado es el mismo. ¿A qué puede deberse que los cambios en el XML no se reflejen?

Respuesta obtenida:

ChatGPT identificó que el módulo estaba siendo editado en /home/operador_odoo/custom_addons/club_padel pero el servicio arrancaba con --addons-path=/opt/odoo/addons,/opt/odoo/custom_addons. Al estar en rutas distintas, Odoo leía siempre la copia antigua.

```
sudo mv /home/operador_odoo/custom_addons/club_padel /opt/odoo/custom_addons/  
sudo -u operador_odoo /usr/bin/python3 /opt/odoo/odoo-bin \  
--addons-path=/opt/odoo/addons,/opt/odoo/custom_addons \  
-d bd_empresa1 -u club_padel --stop-after-init
```

Cómo se aplicó:

Una vez movido el módulo a la ruta correcta y añadiendo -d bd_empresa1, el menú Tienda apareció junto a Pistas y Reservas.

Interacción 3: Arquitectura de módulos - ¿separar tienda en módulo propio?

Prompt utilizado:

Estamos valorando si es mejor tener toda la lógica de tienda dentro del módulo club_padel o crear un módulo independiente llamado tienda_padel. ¿Cuáles son las ventajas e inconvenientes de cada enfoque desde el punto de vista de la arquitectura de Odoo?

Respuesta obtenida:

ChatGPT explicó que separar la tienda sigue el principio de responsabilidad única: permite instalar, actualizar o desactivar la tienda sin tocar la lógica de reservas. Un módulo propio con tienda_padel.producto evita depender de product.template (que requiere el módulo sale). Generó la estructura completa del módulo con modelo, ir.model.access.csv, vistas y menú.

Cómo se aplicó:

Se creó el módulo tienda_padel en /opt/odoo/custom_addons/tienda_padel y se instaló con -i tienda_padel.

Interacción 4: Error al confirmar reservas – columna inexistente en PostgreSQL

Prompt utilizado:

Al intentar confirmar una reserva nos aparece: UndefinedColumn: column club_padel_reserva.pedido_tienda_id does not exist. Tenemos el campo definido en el modelo Python pero parece que la base de datos no lo tiene. ¿Por qué ocurre esto y cómo se sincroniza el esquema de la BD con los modelos en Odoo?

Respuesta obtenida:

ChatGPT explicó que el ORM de Odoo no aplica cambios en el esquema automáticamente al reiniciar el servicio: es necesario ejecutar un upgrade con -u para que Odoo compare el modelo Python con PostgreSQL y cree las columnas. Sin el flag -d bd_empresa1 el upgrade actualizaba una base de datos distinta.

```
sudo -u operador_odoo /usr/bin/python3 /opt/odoo/odoo-bin \
--addons-path=/opt/odoo/addons,/opt/odoo/custom_addons \
-d bd_empresa1 -u club_padel --stop-after-init
```

Cómo se aplicó:

El upgrade con la base de datos correcta creó la columna en PostgreSQL y el error desapareció.

Interacción 5: Diseño del módulo puente entre reservas y tienda

Prompt utilizado:

Queremos conectar las reservas del club con los productos de la tienda. ¿Cuál sería la forma correcta de hacerlo en Odoo sin crear dependencias circulares entre módulos? ¿Tendría sentido crear un tercer módulo que dependa de los dos?

Respuesta obtenida:

ChatGPT confirmó que el patrón correcto es crear un tercer módulo 'puente' que declare los dos como dependencias. Así club_padel y tienda_padel permanecen desacoplados y el módulo club_padel_tienda es el único que conoce a ambos. Generó los modelos CarritoReserva (Many2one a reserva) y CarritoReservaLinea (subtotal computed con @api.depends).

Cómo se aplicó:

Se creó el módulo club_padel_tienda y se instaló con -i club_padel_tienda. El menú 'Carritos por Reserva' quedó visible dentro de la app Tienda Pádel.

Interacción 6: Comprensión de la arquitectura para la defensa

Prompt utilizado:

Necesitamos entender bien la arquitectura del proyecto antes de presentarlo. ¿Puedes explicarnos el papel exacto de cada carpeta del módulo (models, views, security, data, report) y cómo se relacionan entre sí dentro del flujo de Odoo?

Respuesta obtenida:

ChatGPT explicó la arquitectura MVC: models/ contiene las clases Python con la lógica de negocio; views/ contiene los XML con formularios, listas y menús; security/ controla los permisos; data/ carga registros iniciales; report/ define plantillas QWeb para PDFs. El flujo es: menú → acción → vista → modelo → ORM → PostgreSQL. También aclaró la diferencia entre _inherit (ampliar modelo sin tabla nueva) y _name (crear modelo con tabla propia).

Cómo se aplicó:

Esta explicación sirvió de base para la exposición oral y para justificar las decisiones de diseño ante el profesor.

Claude Code (Anthropic) - Fase de mejoras (Mayo 2026, tercer trimestre)

Herramienta: Claude Code CLI (modelo claude-sonnet-4-6) — Mejoras pedidas por el profesor en la corrección

Mejora 1: Módulo mail - chatter y seguimiento de cambios en reservas

Prompt utilizado:

```
"Quiero añadir el chatter de Odoo a las reservas para que queden registrados los cambios de estado y se puedan dejar notas. ¿Qué hay que hacer?"
```

Respuesta obtenida:

Claude Code explicó que Odoo tiene un módulo llamado mail que incluye dos mixins: mail.thread añade el panel de mensajes y mail.activity.mixin añade las actividades programadas. Hay que hacer tres cosas: (1) añadir "mail" a la lista depends del manifest, (2) añadir _inherit = ["mail.thread", "mail.activity.mixin"] en el modelo Python, y (3) añadir el bloque <div class="oe_chatter"> en la vista XML. También añadió tracking=True al campo estado para que cada cambio quede registrado automáticamente.

Código generado (reserva_padel.py):

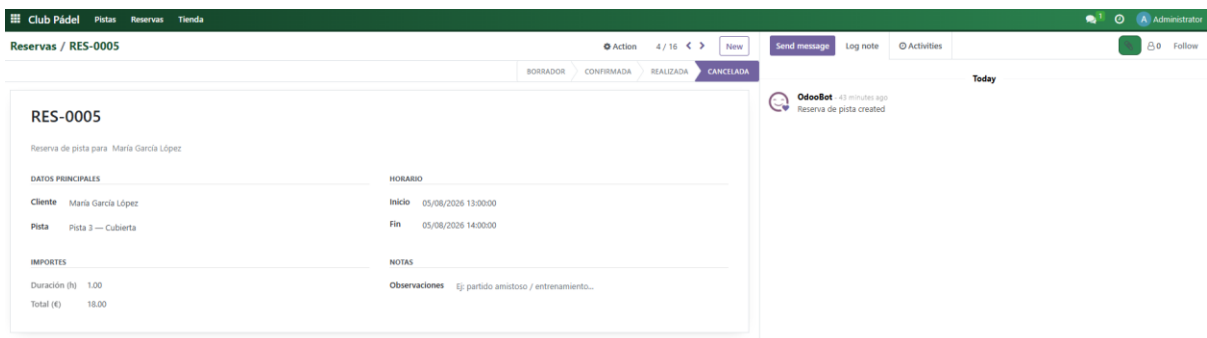
```
class ReservaPadel(models.Model):
    _name = "club_padel.reserva"
    _description = "Reserva de pista"
    _inherit = ["mail.thread", "mail.activity.mixin"]

    estado = fields.Selection(
        [...],
        tracking=True,    # registra cada cambio en el chatter
    )
```

```
<div class="oe_chatter">
  <field name="message_follower_ids"/>
  <field name="activity_ids"/>
  <field name="message_ids"/>
</div>
```

Cómo se aplicó:

Se modificaron `__manifest__.py`, `reserva_padel.py` y `reservas_vistas.xml`. Al abrir una reserva en Odoo ahora aparece el panel de mensajes en la parte inferior, y cada cambio de estado queda registrado automáticamente.



Mejora 2: @api.onchange - autocompletar observaciones al elegir pista

Prompt utilizado:

"Quiero que cuando el usuario seleccione una pista en el formulario de reserva, se rellene solo el campo de observaciones con el tipo y precio de esa pista. ¿Qué decorador uso?"

Respuesta obtenida:

Se usó `@api.onchange`, que es diferente a `@api.depends`: el `onchange` se ejecuta en tiempo real mientras el usuario rellena el formulario (antes de guardar), mientras que `@api.depends` se ejecuta al guardar. Es el tercer decorador del proyecto junto con `@api.constrains`.

Código generado:

```
@api.onchange("pista_id")
def _onchange_pista_id(self):
    if self.pista_id and not self.observaciones:
        self.observaciones = (
            f"Pista {self.pista_id.nombre} "
```

```

        f"({self.pista_id.tipo_pista}) - "
        f"{self.pista_id.precio_hora:.2f)€/h"
    )

```

Cómo se aplicó:

Se añadió el método a reserva_padel.py. Cuando el usuario selecciona una pista y el campo observaciones está vacío, se rellena automáticamente con el nombre, tipo (indoor/outdoor) y precio por hora.

Mejora 3: Vistas graph y pivot - análisis de reservas

Prompt utilizado:

"Añade al módulo de reservas una vista de gráfico de barras y una tabla dinámica para ver los ingresos y horas por pista y por estado."

Respuesta obtenida:

En Odoo, las vistas graph y pivot son tipos de vista estándar que se definen en XML. El graph genera gráficos de barras, líneas o tarta. El pivot genera una tabla dinámica donde los campos type="row" son filas, type="col" son columnas y type="measure" son los valores numéricos.

Código generado (reservas_vistas.xml):

```

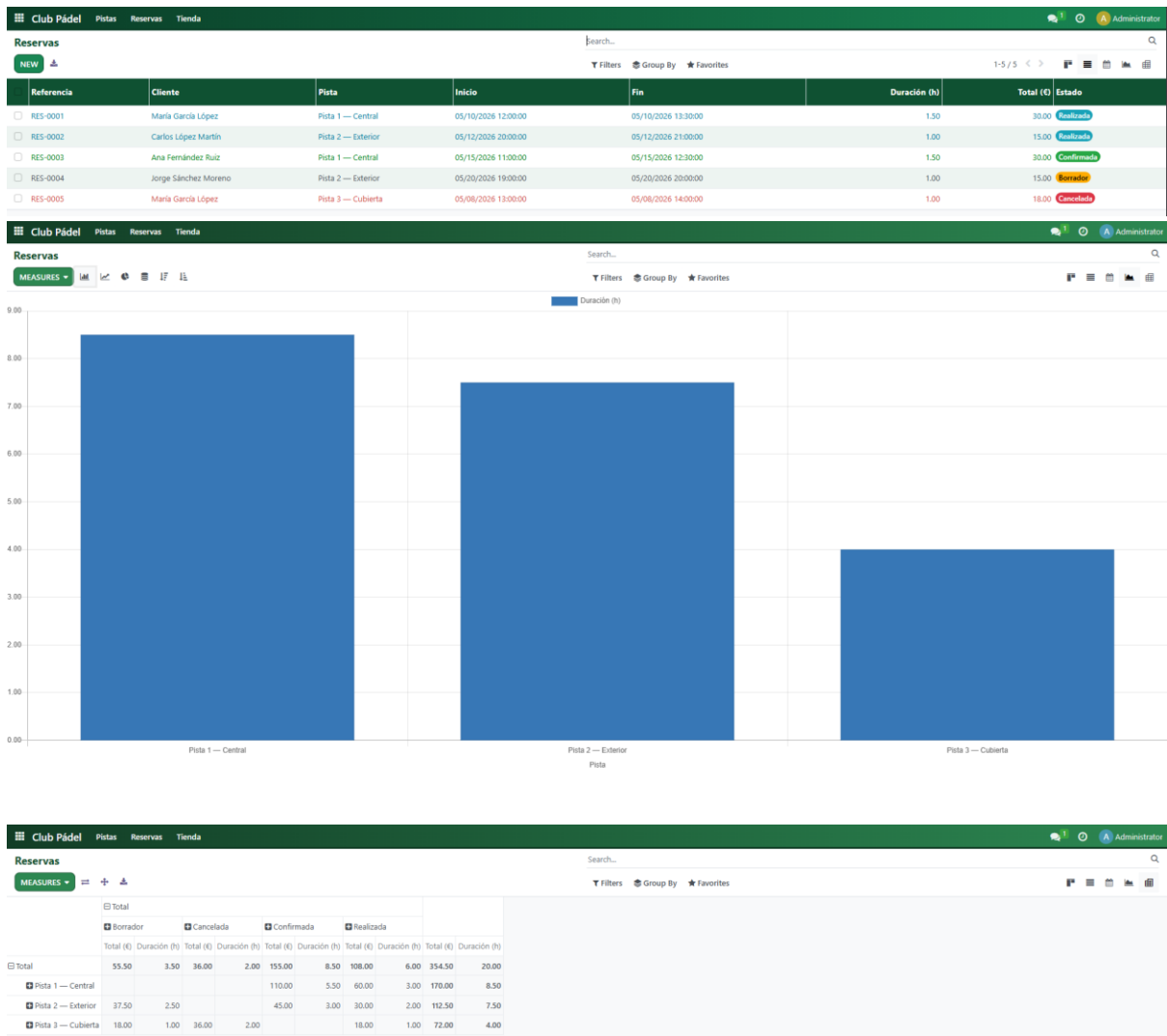
<graph string="Ingresos por pista" type="bar">
    <field name="pista_id" type="row"/>
    <field name="total" type="measure"/>
</graph>

<pivot string="Análisis de reservas">
    <field name="pista_id" type="row"/>
    <field name="estado" type="col"/>
    <field name="total" type="measure"/>
    <field name="duracion_horas" type="measure"/>
</pivot>

```

Cómo se aplicó:

Se añadieron las dos vistas en reservas_vistas.xml y se actualizó la acción a view_mode="tree,form,calendar,graph,pivot". Ahora hay 5 formas de ver los datos de reservas.



Mejora 4: Campos computed avanzados en pista_padel.py

Prompt utilizado:

"Añade al modelo pista_padel los campos iluminacion, superficie, etiqueta_pista (computed) y reservas_activas (computed)"

Código generado:

```
@api.depends("nombre", "tipo_pista", "precio_hora")
def _compute_etiqueta_pista(self):
    for rec in self:
        tipo = dict(rec._fields["tipo_pista"].selection).get(rec.tipo_pista,
        "")
        rec.etiqueta_pista = f"{rec.nombre} · {tipo} ·
        {rec.precio_hora:.2f}€/h"

@api.depends("reserva_ids", "reserva_ids.estado")
```

```
def _compute_reservas_activas(self):
    for rec in self:
        rec.reservas_activas = len(
            rec.reserva_ids.filtered(lambda r: r.estado != "cancelada")
        )
```

Mejora 5: Campo computed ingresos_totales en pista

Prompt utilizado:

Añade a la pista un campo que sume el total de todas sus reservas que estén en estado "realizada".

Respuesta obtenida:

Se añadió un campo Float computed que usa .filtered() para seleccionar solo las reservas realizadas y .mapped() para extraer los totales antes de sumarlos con sum().

```
@api.depends("reserva_ids", "reserva_ids.total", "reserva_ids.estado")
def _compute_ingresos_totales(self):
    for rec in self:
        rec.ingresos_totales = sum(
            rec.reserva_ids.filtered(lambda r: r.estado ==
"realizada").mapped("total")
        )
```

Mejora 6: Vista Kanban de reservas con colores por estado

Prompt utilizado:

Añade una vista kanban a las reservas agrupada por estado, con tarjetas de color diferente según el estado (borrador, confirmada, realizada, cancelada).

Respuesta obtenida:

En Odoo, las tarjetas kanban se colorean usando t-attf-class en el template QWeb, que construye el nombre de una clase CSS de forma dinámica incluyendo el valor del campo estado ({record.estado.raw_value}). En el CSS se define el color para cada clase. Se añade kanban al view_mode de la acción.

```
<div t-attf-class="oe_kanban_card oe_kanban_global_click
        padel_reserva_card padel_estado_#{record.estado.raw_value}">

.padel_estado_borrador { border-left: 5px solid #adb5bd; }
.padel_estado_confirmada { border-left: 5px solid #198754; }
.padel_estado_realizada { border-left: 5px solid #0d6efd; }
.padel_estado_cancelada { border-left: 5px solid #dc3545; }
```

Mejora 7: Datos demo — socios, proveedores, pistas y reservas

Prompt utilizado:

Añade datos demo realistas: 3 proveedores de material de pádel, 4 socios con sus datos de jugador, 3 pistas y 5 reservas de ejemplo con distintos estados.

Respuesta obtenida:

En Odoo, los datos de ejemplo se definen en archivos XML con `noupdate="1"` dentro de `<data>`, lo que significa que se cargan solo la primera vez y no se sobrescriben en actualizaciones. Los campos custom también se rellenan en el XML de demo:

```
<record id="socio_maria_garcia" model="res.partner">
    <field name="name">María García López</field>
    <field name="es_jugador_padel">True</field>
    <field name="nivel_padel">avanzado</field>
    <field name="marca_pala">bullpadel</field>
```

Problema técnico: Caché de assets CSS en Odoo

Prompt utilizado:

He cambiado el CSS pero en el navegador sigue saliendo el estilo antiguo aunque haga Ctrl+Shift+R.

Respuesta obtenida:

En Odoo los archivos CSS se combinan en un bundle guardado en la base de datos como `ir.attachment`. El `Ctrl+Shift+R` solo borra la caché del navegador, no el bundle. La solución es borrarlo con SQL y reiniciar:

```
DELETE FROM ir_attachment WHERE url LIKE '/web/assets/%';  
docker compose restart odoo
```

Mejora 8: Mejoras CSS — colores kanban, lista y barra de navegación

Prompt utilizado:

Mejora el aspecto visual del módulo: colores en las tarjetas kanban según el estado, colores en la lista y tema verde en la barra de navegación.

Respuesta obtenida:

Las filas de la lista se colorean con los atributos decoration-success, decoration-info, decoration-danger en la etiqueta <tree>. Los estilos del backend se añaden en web.assets_backend y sobrescriben las clases CSS de Odoo con !important:

```
<!-- En la vista lista -->  
<tree decoration-success="estado == 'confirmada'"  
      decoration-info="estado == 'realizada'"  
      decoration-danger="estado == 'cancelada'">
```

Claude Code - Fase de corrección de errores (Mayo 2026)

Se utilizó Claude Code para resolver los errores que impedían arrancar los módulos en Docker. A continuación, se documentan los problemas reales y su resolución:

Problema 1: ImportError — archivo cliente_contacto.py inexistente

Prompt utilizado:

"El módulo club_padel no arranca, da este error:
ImportError: cannot import name cliente_contacto from partially initialized module"

Respuesta obtenida:

La IA identificó que `models/__init__.py` importaba `cliente_contacto` pero el archivo no existía. Creó el archivo completo con la herencia de `res.partner` incluyendo campos de jugador de pádel y dos campos computed.

```
class ClienteContacto(models.Model):
    _inherit = "res.partner"

    es_jugador_padel = fields.Boolean(string="Es jugador/a de pádel",
default=False)
    nivel_padel = fields.Selection([
        ("iniciante", "Iniciante"),
        ("intermedio", "Intermedio"),
        ("avanzado", "Avanzado"),
        ("competicion", "Competición"),
    ], string="Nivel de pádel")

    @api.depends("nivel_padel", "marca_pala", "es_jugador_padel", "name")
    def _compute_descripcion_jugador(self):
        for rec in self:
            if rec.es_jugador_padel:
                nivel = dict(rec._fields["nivel_padel"].selection).get(...)
                rec.descripcion_jugador = f"{rec.name} · {nivel} · ..."
```

Problema 2: No matching record found - model_club_padel_tienda_producto

Prompt utilizado:

```
"Nuevo error: No matching record found for external id
model_club_padel_tienda_producto"
```

Respuesta obtenida:

La IA identificó que el modelo `club_padel.tienda_producto` existía en `tienda_producto.py` pero no estaba importado en `models/__init__.py`. Añadió la línea `from . import tienda_producto` para registrar el modelo en Odoo.

Problema 3: FileNotFoundError — archivos de demo inexistentes

Prompt utilizado:

```
"Error: FileNotFoundError: File not found: club_padel/data/demo.xml"
```

Respuesta obtenida:

El manifest referenciaba data/demo.xml y data/productos_demo.xml que no existían. Se eliminaron esas líneas del __manifest__.py.

Problema 4: Atributo name obligatorio en filtros de búsqueda

Prompt utilizado:

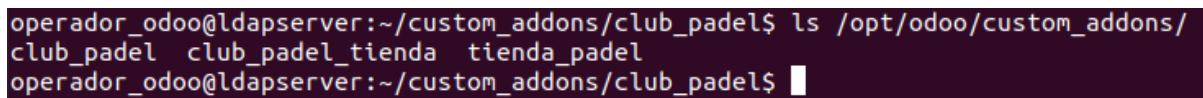
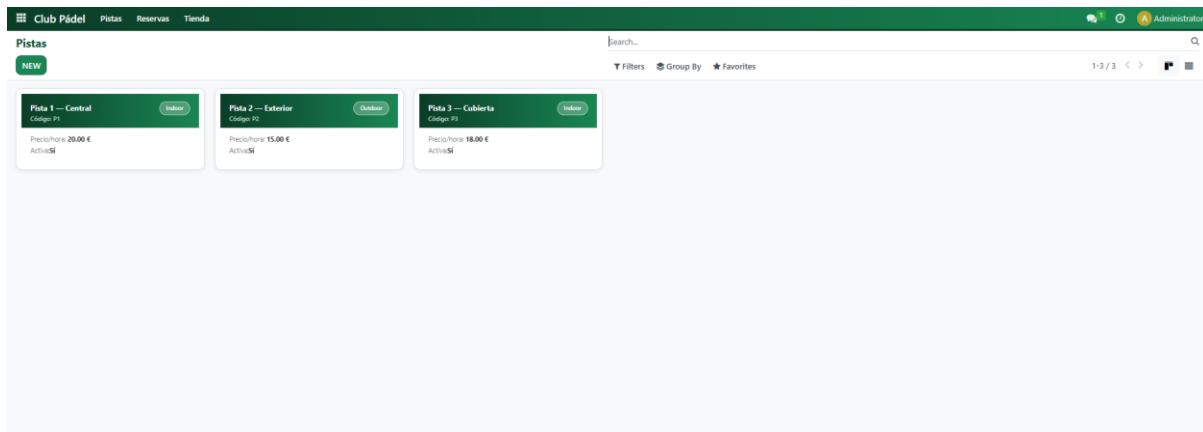
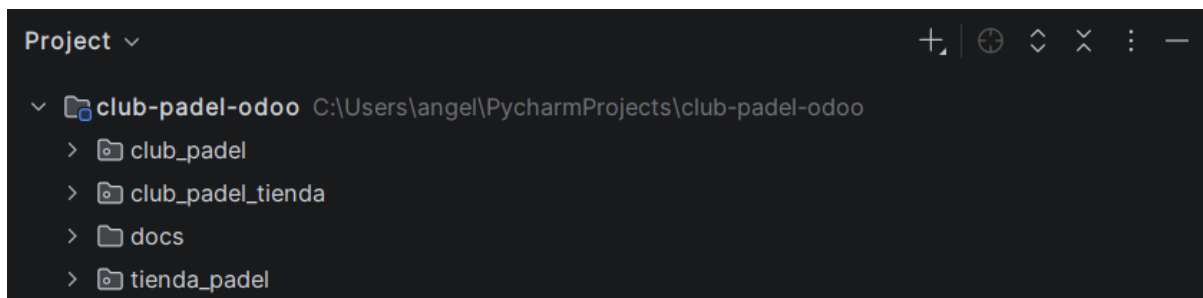
```
"Error al cargar pistas_vistas.xml: Element filter failed to validate attributes"
```

Respuesta obtenida:

En Odoo 16, todos los elementos <filter> de una vista de búsqueda requieren el atributo name, incluyendo los de agrupación dentro de <group>. Se añadieron los atributos faltantes.

2.3 Pasos realizados

1. Instalación de Odoo 16 Community en servidor local Linux (Ubuntu 22).
2. Creación de carpeta personalizada para addons: /opt/odoo/custom_addons/
3. Desarrollo de tres módulos propios: club_padel, tienda_padel y club_padel_tienda.
4. Definición de modelos personalizados en Python siguiendo la arquitectura MVC de Odoo.
5. Creación de vistas XML personalizadas (formularios, listas, calendar, graph, pivot y menús).
6. Configuración de permisos de acceso mediante ir.model.access.csv.
7. Integración de estilos CSS personalizados en web.assets_backend.
8. Actualización de módulos mediante: `odoo-bin -d bd_empresa1 -u nombre_modulo`
9. Reinicio del servicio Odoo y validación funcional de cada módulo.
10. Backup de base de datos mediante el gestor de bases de datos de Odoo (formato .zip).



2.4 Códigos y módulos creados

Módulo: club_padel

Módulo principal. Responsable de la gestión del club deportivo: pistas, reservas y clientes.

- Gestión de pistas de pádel con campos avanzados (iluminación, superficie, etiqueta computed).
- Gestión de reservas horarias con validación de solapamientos.
- Integración con el chatter (módulo mail) para trazabilidad completa.
- Vistas graph y pivot para análisis de datos.
- Informe PDF de confirmación de reserva (QWeb).
- Herencia de res.partner para añadir perfil de jugador de pádel.

Club Pádel Pistas Reservas Tienda

Reservas / New

CONFIRMAR CANCELAR BORRAR CONFIRMADA REALIZADA CANCELADA

Nuevo

Reserva de pista para

DATOS PRINCIPALES		HORARIO	
Cliente		Inicio	
Pista		Fin	

IMPORTES		NOTAS	
Duración (h)	0.00	Observaciones	Ej: partido amistoso / entrenamiento...
Total (€)	0.00		

Administrator Creating a new record...

Today

```

operador_odoo@ldapserver:~/custom_addons/club_padel$ sudo tree -L 2 /opt/odoo/custom_addons/club_padel
/opt/odoo/custom_addons/club_padel
├── data
│   ├── demo.xml
│   ├── productos_demo.xml
│   ├── secuencia_ventas.xml
│   ├── secuencia.xml
│   ├── tienda_data.xml
│   └── tienda_productos_demo.xml
├── __init__.py
├── __manifest__.py
├── models
│   ├── cliente_contacto.py
│   ├── __init__.py
│   ├── pedido_tienda.py
│   ├── pista_padel.py
│   ├── producto_padel.py
│   ├── __pycache__
│   ├── reserva_padel.py
│   ├── tienda_producto.py
│   └── venta_tienda.py
├── __pycache__
│   └── __init__.cpython-310.pyc
├── report
│   ├── plantilla_reporte_reserva.xml
│   └── reporte_reserva.xml
├── security
│   └── ir.Model.access.csv
├── static
│   ├── description
│   └── src
└── views
    ├── clientes_vistas.xml
    ├── menu.xml
    ├── pistas_vistas.xml
    ├── productos_vistas.xml
    ├── reserva_extension_tienda.xml
    ├── reservas_vistas.xml
    ├── tienda_productos_action.xml
    ├── tienda_producto_views.xml
    └── tienda_vistas.xml

10 directories, 29 files
operador_odoo@ldapserver:~/custom_addons/club_padel$

```

Módulo: tienda_padel

Módulo de gestión de productos deportivos del club:

- Modelo propio tienda_padel.producto con campos: nombre, categoría (palas/pelotas/textil/accesorios), precio, stock, imagen.
- Vistas de lista y formulario personalizadas.
- Menú propio en el backend.

```
operador_odoo@ldapserver:~/custom_addons/club_padel$ sudo tree -L 2 /opt/odoo/custom_addons/tienda_padel
/opt/odoo/custom_addons/tienda_padel
├── data
│   └── demo.xml
├── __init__.py
├── manifest__.py
├── models
│   ├── __init__.py
│   ├── __pycache__
│   └── tienda_producto.py
├── __pycache__
│   └── __init__.cpython-310.pyc
├── security
│   └── ir.model.access.csv
├── static
│   └── description
└── views
    ├── menu.xml
    └── tienda_producto_views.xml

8 directories, 9 files
operador_odoo@ldapserver:~/custom_addons/club_padel$
```

Módulo: club_padel_tienda

Módulo puente que integra reservas con productos mediante un sistema de carrito:

- Modelo CarritoReserva: vincula una reserva con productos de la tienda.
- Modelo CarritoReservaLinea: líneas del carrito (producto, cantidad, precio, subtotal computed).
- Total del carrito calculado automáticamente mediante @api.depends.
- Relaciones Many2one y One2many entre módulos sin dependencias circulares.

Tienda Pádel Productos Carritos por Reserva

Carritos por reserva / New

Carrito Carrito - Reserva 1

Reserva RES-0001

Total (€) 0.00

Productos

Producto	Cantidad	Precio (€)	Subtotal...
Add a line			

```

operator_odoo@ldapserver:~/custom_addons/club_padel$ sudo tree -L 2 /opt/odoo/custom_addons/club_padel_tienda
/opt/odoo/custom_addons/club_padel_tienda
├── __init__.py
├── __manifest__.py
├── models
│   ├── carrito.py
│   ├── __init__.py
│   └── __pycache__
├── __pycache__
│   └── __init__.cpython-310.pyc
├── security
│   └── ir.model.access.csv
└── views
    ├── carrito_views.xml
    └── menu.xml

5 directories, 8 files
operator_odoo@ldapserver:~/custom_addons/club_padel$

```

2.5 Modelos

Modelo de pistas (club_padel)

```
GNU nano 6.2 club_padel/models/pista_padel.py
from odoo import models, fields, api
from odoo.exceptions import ValidationError

class PistaPadel(models.Model):
    _name = "club_padel.pista"
    _description = "Pista de pádel"
    _rec_name = "nombre"

    nombre = fields.Char(string="Nombre", required=True)
    codigo = fields.Char(string="Código", required=True)
    activa = fields.Boolean(default=True)

    tipo_pista = fields.Selection(
        [("indoor", "Indoor"), ("outdoor", "Outdoor")],
        string="Tipo de pista",
        default="indoor",
        required=True,
    )

    precio_hora = fields.Float(string="Precio por hora (€)", required=True, default=18.0)
    notas = fields.Text(string="Notas")

    _sql_constraints = [
        ("codigo_unico", "unique(codigo)", "El código de pista ya existe."),
    ]

    @api.constrains("precio_hora")
    def _comprobar_precio(self):
        for rec in self:
            if rec.precio_hora < 0:
                raise ValidationError("El precio por hora no puede ser negativo.")
```

Modelo de reservas (club_padel):

```
GNU nano 6.2 club_padel/models/reserva_padel.py

from odoo import models, fields, api
from odoo.exceptions import ValidationError

class ReservaPadel(models.Model):
    _name = "club_padel.reserva"
    _description = "Reserva de pista"
    _rec_name = "referencia"

    referencia = fields.Char(string="Referencia", required=True, copy=False, readonly=True, default="")

    cliente_id = fields.Many2one("res.partner", string="Cliente", required=True)
    pista_id = fields.Many2one("club_padel.pista", string="Pista", required=True)

    inicio = fields.Datetime(string="Inicio", required=True)
    fin = fields.Datetime(string="Fin", required=True)

    telefono_contacto = fields.Char(string="Teléfono", related="cliente_id.phone", readonly=True)
    email_contacto = fields.Char(string="Email", related="cliente_id.email", readonly=True)

    estado = fields.Selection(
        [
            ("borrador", "Borrador"),
            ("confirmada", "Confirmada"),
            ("realizada", "Realizada"),
            ("cancelada", "Cancelada"),
        ],
        string="Estado",
        default="borrador",
        required=True,
    )

    duracion_horas = fields.Float(string="Duración (h)", compute="_calcular_duracion", store=True)
    total = fields.Float(string="Total (€)", compute="_calcular_total", store=True)

    observaciones = fields.Text(string="Observaciones")

    @api.model
    def create(self, vals):
        if vals.get("referencia", "Nuevo") == "Nuevo":
            vals["referencia"] = self.env["ir.sequence"].next_by_code("club_padel.reserva.seq")
        return super().create(vals)

    @api.depends("inicio", "fin")
    def _calcular_duracion(self):
        for rec in self:
            if rec.inicio and rec.fin:
                delta = rec.fin - rec.inicio
                rec.duracion_horas = delta.total_seconds() / 3600.0
            else:
                rec.duracion_horas = 0

    @api.depends("duracion_horas", "pista_id.precio_hora")
    def _calcular_total(self):
        for rec in self:
            rec.total = rec.duracion_horas * (rec.pista_id.precio_hora or 0)

    @api.constrains("inicio", "fin")
    def _comprobar_fechas(self):
        for rec in self:
            if rec.inicio and rec.fin and rec.fin <= rec.inicio:
                raise ValidationError("La fecha/hora de fin debe ser posterior al inicio.")

    @api.constrains("pista_id", "inicio", "fin", "estado")
    def _comprobar_solape(self):
        for rec in self:
            if not rec.pista_id or not rec.inicio or not rec.fin:
                continue
            if rec.estado == "cancelada":
                continue

            dominio = [
                ("id", "!=", rec.id),
                ("pista_id", "=", rec.pista_id.id),
                ("estado", "!=", "cancelada"),
                ("inicio", "<", rec.fin),
                ("fin", ">", rec.inicio),
            ]
            if self.search_count(dominio) > 0:
                raise ValidationError("Ya existe una reserva que se solapa en esa pista y horario")
```

```
# --- BOTONES DEL FORMULARIO (coinciden con la vista) ---
def accion_confirmar(self):
    for rec in self:
        rec.estado = "confirmada"

def accion_realizada(self):
    for rec in self:
        rec.estado = "realizada"

def accion_cancelar(self):
    for rec in self:
        rec.estado = "cancelada"

def accion_imprimir_confirmacion(self):
    accion = self.env.ref("club_padel.accion_reporte_reserva")
    return accion.report_action(self)
```

Modelo productos tienda (tienda_padel):

```
GNU nano 6.2 tienda_padel/models/tienda_producto.py
from odoo import models, fields

class TiendaPadelProducto(models.Model):
    _name = "tienda_padel.producto"
    _description = "Producto Tienda Pádel"
    _rec_name = "nombre"

    nombre = fields.Char(string="Nombre", required=True)
    categoria = fields.Selection(
        [
            ("palas", "Palas"),
            ("pelotas", "Pelotas"),
            ("textil", "Textil"),
            ("accesorios", "Accesorios"),
        ],
        string="Categoría",
        default="accesorios",
        required=True
    )
    precio = fields.Float(string="Precio (€)", default=0.0)
    stock = fields.Integer(string="Stock", default=0)
    activo = fields.Boolean(string="Activo", default=True)
    descripcion = fields.Text(string="Descripción")
    imagen = fields.Image(string="Imagen")
```

Modelo carrito integración (club_padel_tienda):

```
GNU nano 6.2 club_padel_tienda/models/carrito.py
from odoo import models, fields, api

class ClubPadelTiendaCarrito(models.Model):
    _name = "club_padel_tienda.carrito"
    _description = "Carrito de tienda para una reserva"
    _rec_name = "name"

    name = fields.Char(string="Carrito", compute="_compute_name", store=True)
    reserva_id = fields.Many2one("club_padel.reserva", string="Reserva", required=True, ondelete="set null")

    linea_ids = fields.One2many(
        "club_padel_tienda.carrito_linea",
        "carrito_id",
        string="Lineas"
    )

    total = fields.Float(string="Total (€)", compute="_compute_total", store=True)

    @api.depends("reserva_id")
    def _compute_name(self):
        for rec in self:
            # No asumimos el nombre de campo de la reserva (puede ser referencia, name, etc.)
            # Mostramos "Reserva <id>" para no fallar nunca.
            rec.name = f"Carrito - Reserva {rec.reserva_id.id}" if rec.reserva_id else "Carrito"

    @api.depends("linea_ids.subtotal")
    def _compute_total(self):
        for rec in self:
            rec.total = sum(rec.linea_ids.mapped("subtotal"))

class ClubPadelTiendaCarritoLinea(models.Model):
    _name = "club_padel_tienda.carrito_linea"
    _description = "Línea de carrito"

    carrito_id = fields.Many2one("club_padel_tienda.carrito", string="Carrito", required=True, ondelete="set null")
    producto_id = fields.Many2one("tienda_padel.producto", string="Producto", required=True)

    cantidad = fields.Float(string="Cantidad", default=1.0)
    precio = fields.Float(string="Precio (€)", related="producto_id.precio", store=True, readonly=True)
    subtotal = fields.Float(string="Subtotal (€)", compute="_compute_subtotal", store=True)

    @api.depends("cantidad", "precio")
    def _compute_subtotal(self):
        for line in self:
            line.subtotal = (line.cantidad or 0.0) * (line.precio or 0.0)
```

Vistas XML personalizadas

- Formularios personalizados.
- Listas adaptadas.
- Menús propios en el backend.
- Modificación de vistas existentes mediante herencia.

CSS personalizado

Archivo:

`static/src/css/club_padel.css`

Incluye:

- Cambio de color de navbar.
- Estilo personalizado para botones.
- Cabeceras de tabla modificadas.
- Fondo corporativo.

2.6 Backup de la base de datos

Se realizó copia de seguridad mediante:

- Odoo backend (gestor de bases de datos).
- Exportación en formato `.zip`.

Contraseñas necesarias:

- Usuario administrador Odoo.
- Usuario PostgreSQL.

[bd_empresa1_2026-02-24_19-14-22](#)

3. Customizaciones realizadas

3.1 Modificación de campos y vistas

- Añadido campo Estado en modelo de pistas con opciones Activa/Mantenimiento/Cerrada.
- Añadidos campos iluminacion (Boolean) y superficie (Selection: cristal/moqueta/cesped) al modelo de pistas.
- Campo computed etiqueta_pista que muestra nombre + tipo + precio en una sola línea.
- Campo computed reservas_activas que cuenta en tiempo real las reservas no canceladas de una pista.
- Modificación de formularios con agrupación visual de campos por secciones.
- Personalización de listas con filtros y agrupaciones.

```
operador_odoo@ldapserver:~/custom_addons$ grep -n "fields" /opt/odoo/custom_addons/  
club_padel/models/pista_padel.py  
1:from odoo import models, fields, api  
9:     nombre = fields.Char(string="Nombre", required=True)  
10:     codigo = fields.Char(string="Código", required=True)  
11:     activa = fields.Boolean(default=True)  
13:     tipo_pista = fields.Selection(  
20:     precio_hora = fields.Float(string="Precio por hora (€)", required=True, default=18.0)  
21:     notas = fields.Text(string="Notas")  
operador_odoo@ldapserver:~/custom_addons$
```

3.2 Gestión de clientes, proveedores y productos

En lugar de crear modelos propios paralelos, se optó por **ampliar los modelos nativos de Odoo** mediante herencia (`_inherit`), integrando los datos del club directamente en el ERP existente. Esta decisión es la arquitectura correcta en Odoo: evita duplicar datos y permite que los módulos estándar (Contacts, Sales, Inventory) sigan funcionando con los datos del club.

Ampliación del modelo de productos (`product.template`)

El archivo `producto_padel.py` hereda el modelo nativo `product.template` de Odoo para añadir campos específicos de pádel sin crear una tabla nueva:

```
class ProductoPadel(models.Model):
    _inherit = "product.template"

    es_producto_padel = fields.Boolean(string="Producto de pádel", default=False)
    tipo_producto_padel = fields.Selection(
        [("pala", "Pala"), ("pelotas", "Pelotas"),
         ("grip", "Grip/Overgrip"), ("textil", "Textil"), ("otros", "Otros")],
        string="Tipo de producto (pádel)",
    )
```

Gracias a esto, los productos del club son productos reales de Odoo con `sale_ok=True`, `purchase_ok=True`, `list_price` y `standard_price`, lo que significa que están disponibles en los módulos de Ventas e Inventario sin configuración adicional.

Productos cargados en demo:

Nombre	Precio (€)	Stock	Activo
☐ Pelotas de pádel (bote 3)	8.50	40	
☐ Pala iniciación	59.99	10	
☐ Bulpadel Hack 03	189.99	4	
☐ Overgrip Wilson (pack 3)	5.99	60	
☐ Bolsa de pádel Head Tour	45.00	6	

Ampliación del modelo de contactos (res.partner)

El archivo `cliente_contacto.py` hereda `res.partner`, el modelo nativo de clientes y proveedores de Odoo, añadiendo el perfil de jugador de pádel:

```
class ClienteContacto(models.Model):
    _inherit = "res.partner"

    es_jugador_padel = fields.Boolean(...)
    nivel_padel = fields.Selection(...)
    mano_dominante = fields.Selection(...)
    marca_pala = fields.Selection(...)
    modelo_pala = fields.Char(...)
    descripcion_jugador = fields.Char(compute="_compute_descripcion_jugador", store=True)
    reserva_ids = fields.One2many("club_padel.reserva", "cliente_id", ...)
    total_reservas = fields.Integer(compute="_compute_total_reservas", store=True)
```

Esto permite que los socios del club sean contactos reales de Odoo, con historial de reservas accesible desde su ficha.

Proveedores cargados en demo:

Contacts

Contacts

Configuration

Contacts

NEW

Bullpadel S.L.

Madrid, Spain

pedidos@bullpadel.com

Dunlop Sports España

Barcelona, Spain

ventas@dunlop.es

Head Sport GmbH

Kennelbach, Austria

info@head.com

Socios cargados en demo:

Contacts

Contacts

Configuration

Contacts

NEW

Individuals

Search...

Filters

Group By

Favorites

1-8 / 8

A

Administrator

admin@example.com

Ana Fernández Ruiz

Valencia, Spain

ana.fernandez@email.com

Carlos López Martín

Valencia, Spain

carlos.lopez@email.com

Jorge Sánchez Moreno

Valencia, Spain

jorge.sanchez@email.com

Laura Pérez Sanz

Valencia, Spain

laura.perez@email.com

Maria García López

Valencia, Spain

maria.garcia@email.com

Miguel Romero Gil

Valencia, Spain

miguel.romero@email.com

Sofía Torres Vega

Valencia, Spain

sofia.torres@email.com

Ficha de Socio - Pestaña Pádel

Contacts

Contacts

Configuration

Contacts / María García López

Action

8 / 9

New

Valencia

State

ZIP

Mobile

Spain

Email

maria.garcia@email.com

Tax ID [?]

e.g. BE0477472701

Website

e.g. https://www.odoo.com

Title

e.g. Mister

Tags

e.g. "B2B", "VIP", "Consulting", ...

Contacts & Addresses

Sales & Purchase

Internal Notes

Pádel

INFORMACIÓN DEL JUGADOR

Es jugador/a de pádel ☒

Nivel de pádel

Avanzado

Mano dominante

Diestra

Marca de la pala

Bullpadel

Modelo de la pala

Hack 03

RESUMEN AUTOMÁTICO

Descripción del jugador

María García López · Avanzado · Bullpadel

Total de reservas

2

RESERVAS DEL SOCIO

Referen...	Pista	Inicio
RES-0001	Pista 1 — Ce...	05/10/20...
RES-0005	Pista 3 — Cu...	05/08/20...

Add a line

3.3 Creación de módulo propio

Se desarrollaron tres módulos personalizados con estructura modular que evita dependencias circulares:

Módulo	Dependencias	Responsabilidad
club_padel	base, contacts, product, web, mail	Pistas, reservas, clientes, informes
tienda_padel	base, web	Productos deportivos de la tienda
club_padel_tienda	club_padel, tienda_padel	Carrito vinculado a reservas

club_padel/__manifest__.py

```
GNU nano 6.2 /opt/odoo/custom_addons/club_padel/__manifest__.py
{
    "name": "Club Pádel",
    "version": "16.0.1.0.0",
    "category": "Services",
    "summary": "Gestión de pistas, reservas y tienda para un club de pádel",
    "author": "Rubén y Ángel",
    "depends": ["base", "contacts", "product", "web"],
    "data": [
        # =====
        # SEGURIDAD
        # =====
        "security/ir.model.access.csv",

        # =====
        # DATOS / DEMO / SECUENCIAS
        # =====
        "data/secuencia.xml",
        "data/secuencia_ventas.xml",
        "data/demo.xml",
        "data/productos_demo.xml",
        "data/tienda_productos_demo.xml",

        # =====
        # REPORTES
        # =====
        "report/reporte_reserva.xml",

        # =====
        # VISTAS PRINCIPALES
        # =====
        "views/pistas_vistas.xml",
        "views/reservas_vistas.xml",
        "views/clientes_vistas.xml",
        "views/productos_vistas.xml",

        # =====
        # TIENDA (NUEVO)
        # =====
        "views/tienda_producto_views.xml",
        "views/tienda_productos_action.xml",
    ]
}
```

```

# =====
# MENÚ (SIEMPRE AL FINAL)
# =====
"views/menu.xml",
],
"assets": {
    "web.assets_backend": [
        "club_padel/static/src/css/club_padel.css",
    ],
},
"application": True,
"installable": True,
"license": "LGPL-3",
}

```

tienda_padel/__manifest__.py

```

GNU nano 6.2 /opt/odoo/custom_addons/tienda_padel/ __manifest__.py
{
    "name": "Tienda Pádel",
    "version": "16.0.1.0.0",
    "category": "Sales",
    "summary": "Gestión de productos para una tienda de pádel",
    "author": "Rubén y Ángel",
    "depends": ["base", "web"],
    "data": [
        "security/ir.model.access.csv",
        "views/tienda_producto_views.xml",
        "views/menu.xml",
        "data/demo.xml",
    ],
    "application": True,
    "installable": True,
    "license": "LGPL-3",
}

```

club_padel_tienda/__manifest__.py

```

GNU nano 6.2 /opt/odoo/custom_addons/club_padel_tienda/ __manifest__.py
{
    "name": "Club Pádel - Tienda (Integración)",
    "version": "16.0.1.0.0",
    "category": "Services",
    "summary": "Conecta reservas del club con productos de la tienda",
    "author": "Rubén y Ángel",
    "depends": ["base", "club_padel", "tienda_padel"],
    "data": [
        "security/ir.model.access.csv",
        "views/carrito_views.xml",
        "views/menu.xml",
    ],
    "application": False,
    "installable": True,
    "license": "LGPL-3",
}

```

Funcionalidades:

- Gestión de pistas.
- Modelo personalizado.
- Menú propio en el backend.
- Integración con clientes.
- Estética personalizada.

Por qué se crearon módulos propios en lugar de ampliar los nativos existentes

Los módulos nativos de Odoo como `sale`, `purchase` o `stock` están diseñados para gestión empresarial genérica. Instalarlos arrastra decenas de modelos, vistas y dependencias innecesarias para un club deportivo: facturas, albaranes, rutas de almacén, tarifas de cliente... Usar `_inherit` sobre ellos sin tenerlos instalados es inviable porque los campos dependen de que el módulo base esté activo.

La decisión fue crear modelos propios ligeros para la lógica específica del club y **ampliar únicamente los modelos del núcleo de Odoo** que siempre están disponibles: `product.template` (productos) y `res.partner` (contactos). Así conseguimos integración real con el ERP sin dependencias innecesarias.

Esta arquitectura tiene tres ventajas concretas:

- Los productos de la tienda y los socios son objetos nativos de Odoo, no tablas paralelas.
- El sistema es extensible: conectar con Ventas o Compras en el futuro no requiere migración de datos.
- El código es mantenible e independiente: cada módulo tiene una responsabilidad clara y se puede instalar o desactivar sin afectar al resto.

3.4 Cambios en el entorno gráfico

Se realizaron cambios visuales mediante CSS personalizado registrado en `web.assets_backend`:

- Navbar corporativo con gradiente verde oscuro (`#0f5132` → `#198754`).
- Botones primarios con colores del club.
- Cabeceras de tabla estilizadas.
- Fondo corporativo en formularios.
- Hover en el menú lateral con colores adaptados.

```

/* Barra superior */
.o_main_navbar {
    background: linear-gradient(90deg, #0f5132, #198754) !important;
    border-bottom: 3px solid #0a3d27;
}

/* Botón NEW */
.btn-primary {
    background-color: #198754 !important;
    border-color: #0f5132 !important;
}

/* Tabla estilo deportivo */
.o_list_view thead {
    background-color: #0f5132 !important;
    color: white !important;
}

```

3.5 Mejoras extra realizadas

Tras la corrección inicial, se implementaron las siguientes mejoras adicionales:

- Integración del módulo mail: chatter y tracking de cambios de estado en reservas.
- Implementación de @api.onchange para autocompletar el campo de observaciones al seleccionar una pista.
- Vistas graph (gráfico de barras) y pivot (tabla dinámica) para análisis de reservas por pista, estado, ingresos y horas.
- Campos computed avanzados en pista_padel: etiqueta_pista y reservas_activas.
- Campos de jugador en res.partner: nivel_padel, mano_dominante, marca_pala con visibilidad condicional (attrs).

Informe PDF de confirmación de reserva (QWeb)

Se implementó un informe PDF generado desde Odoo usando el motor de plantillas QWeb. Al confirmar una reserva, el botón "Imprimir confirmación" genera un documento con los datos de la reserva:

```

<template id="plantilla_reporte_reserva">
<t t-call="web.external_layout">
<t t-foreach="docs" t-as="o">
<p><strong>Referencia:</strong> <span t-esc="o.referencia"/></p>
<p><strong>Cliente:</strong> <span t-esc="o.cliente_id.name"/></p>
<p><strong>Pista:</strong> <span t-esc="o.pista_id.nombre"/></p>
<p><strong>Total:</strong> <span t-esc="o.total"/> €</p>

```

```
</t>
</t>
</template>
```

El informe se declara como `ir.actions.report` y se invoca desde el modelo con `report_action(self)`. Es el mismo mecanismo que usan los módulos nativos de Odoo para generar facturas y albaranes.

4. Customizaciones realizadas

4.1 Entorno de Desarrollo

La implantación se realizó en un entorno Linux con las siguientes tecnologías:

Componente	Versión / Detalle
Sistema operativo	Ubuntu 22.04 LTS
ERP	Odoo 16 Community
Base de datos	PostgreSQL (motor nativo de Odoo)
Lenguaje backend	Python 3.10
Puerto de servicio	8069 (por defecto Odoo)
Carpeta de addons	/opt/odoo/custom_addons/
Servicio	odoo16.service (systemd)

4.2 Arquitectura de Odoo

Odoo funciona bajo arquitectura MVC:

- Modelo (Models): Python → definición de datos y lógica de negocio.
- Vista (Views): XML → representación visual (formularios, listas, gráficos).
- Controlador: Gestionado internamente por el ORM de Odoo.

Los tres módulos respetan esta estructura, lo que garantiza escalabilidad, mantenibilidad e integración con módulos estándar.

5. Diseño del Modelo de Datos

5.1 Modelo Principal: Pistas (club_padel.pista)

Campo	Tipo	Descripción
nombre	Char (req)	Nombre de la pista
codigo	Char (req, unique)	Código único de la pista
activa	Boolean	Si la pista está disponible
tipo_pista	Selection	Indoor / Outdoor
precio_hora	Float (req)	Precio por hora en €
iluminacion	Boolean	Si dispone de iluminación artificial
superficie	Selection	Cristal / Moqueta / Césped
etiqueta_pista	Char (computed)	Resumen: nombre · tipo · precio
reservas_activas	Integer (computed)	Nº de reservas no canceladas
notas	Text	Observaciones libres

5.2 Modelo de Reservas (club_padel.reserva)

Campo	Tipo	Descripción
referencia	Char (readonly)	Nº automático de reserva (secuencia)
cliente_id	Many2one → res.partner	Cliente que realiza la reserva
pista_id	Many2one → club_padel.pista	Pista reservada
inicio	Datetime (req)	Fecha y hora de inicio
fin	Datetime (req)	Fecha y hora de fin
estado	Selection (tracking)	Borrador / Confirmada / Realizada / Cancelada
duracion_horas	Float (computed)	Duración calculada en horas
total	Float (computed)	Total = duración × precio/hora

observaciones	Text	Autocompletado con @api.onchange
---------------	------	----------------------------------

5.3 Modelo de Jugador (herencia res.partner)

Campo añadido	Tipo	Descripción
es_jugador_padel	Boolean	Activa los campos de jugador (attrs)
nivel_padel	Selection	Iniciante / Intermedio / Avanzado / Competición
mano_dominante	Selection	Derecha / Izquierda
marca_pala	Char	Marca de pala habitual
descripcion_jugador	Char (computed)	Resumen del perfil del jugador
total_reservas	Integer (computed)	Nº total de reservas del jugador

6. Seguridad y Permisos

Se definieron reglas de acceso mediante security/ir.model.access.csv para cada modelo de los tres módulos:

Modelo	Grupo	Permisos	Restricciones
club_padel.pista	Administrador	CRUD completo	—
club_padel.pista	Usuario básico	Solo lectura	No puede crear/editar
club_padel.reserva	Administrador	CRUD completo	—
club_padel.reserva	Usuario básico	Lectura + crear	No puede borrar
tienda_padel.producto	Administrador	CRUD completo	—

Esta configuración garantiza la integridad de datos, el control interno y la prevención de errores humanos al limitar qué puede hacer cada tipo de usuario.

7. Personalización del Entorno Gráfico

7.1 Gestión de Assets CSS

Se integró CSS personalizado mediante `web.assets_backend` declarado en el **manifest.py** del módulo `club_padel`. Esto garantiza que los estilos afectan únicamente al entorno administrativo (backend), evitando conflictos con el portal web público.

Declaración en manifest.py:

```
"assets": {
    "web.assets_backend": [
        "club_padel/static/src/css/club_padel.css",
    ],
},
```

Se validó la carga correcta del CSS mediante el modo debug de Odoo: ?debug=assets

GNU nano 6.2 /opt/odoo/custom_addons/club_padel/static/src/css/club_padel.css *

```
/* =====
TEMA CLUB PÁDEL
===== */

/* Barra superior */
.o_main_navbar {
    background: linear-gradient(90deg, #0f5132, #198754) !important;
    border-bottom: 3px solid #0a3d27;
}

.o_main_navbar .o_menu_brand,
.o_main_navbar .o_menu_sections a {
    color: #ffffff !important;
    font-weight: 600;
}

/* Menú lateral */
.o_web_client .o_menu_apps {
    background-color: #f4f7f5 !important;
}

.o_web_client .o_menu_apps .o_app {
    color: #0f5132 !important;
    font-weight: bold;
}

/* Hover menú lateral */
.o_web_client .o_menu_apps .o_app:hover {
    background-color: #d1e7dd !important;
    color: #0a3d27 !important;
}

/* Tabla estilo deportivo */
.o_list_view thead {
    background-color: #0f5132 !important;
    color: white !important;
}

.o_list_view tbody tr:nth-child(even) {
    background-color: #e9f5ee !important;
}

/* Botón NEW */
.btn-primary {
    background-color: #198754 !important;
    border-color: #0f5132 !important;
    font-weight: bold;
}

.btn-primary:hover {
    background-color: #0f5132 !important;
}

/* Formularios */
.o_form_view .o_form_sheet_bg {
    background: linear-gradient(180deg, #ffffff, #f1f9f4);
}

/* Títulos */
.o_control_panel .breadcrumb {
    color: #0f5132 !important;
    font-weight: bold;
}
```

```
operador_odoo@ldapserver:~/custom_addons$ grep -n "assets" /opt/odoo/custom_addons/  
club_padel/__manifest__.py  
47:     "assets": {  
48:         "web.assets_backend": [  
operador_odoo@ldapserver:~/custom_addons$
```

8. Metodología de Trabajo

8.1 División del trabajo entre miembros del equipo

Tarea	Responsable
Estructura inicial de los tres módulos	Rubén Flores
Docker Compose para entorno local	Rubén Flores
Crear cliente_contacto.py (herencia res.partner)	Rubén Flores
Corrección de imports y manifest	Rubén Flores
Campos computed en pista_padel.py	Rubén Flores
Ampliar vista de contactos (pestaña Pádel)	Àngel Martínez
Actualizar formulario de pistas con nuevos campos	Àngel Martínez
CSS personalizado y personalización gráfica	Àngel Martínez

Repositorio de commits: <https://github.com/rfloresg/club-padel-odoo/commits/main>

8.2 Valoración del trabajo - Rubén Flores

Autovaloración:

Me encargué de la parte técnica más compleja: diseñar la arquitectura de los tres módulos desde cero, configurar el entorno Docker en Windows y resolver todos los errores de arranque (imports rotos, archivos inexistentes en el manifest, filtros XML sin atributo obligatorio). Implementé toda la lógica Python: campos computed con `@api.depends` en pistas y contactos, validaciones con `@api.constrains` para evitar solapamientos, el decorador `@api.onchange` para autocompletar campos, la integración del módulo mail para el chatter y el campo `ingresos_totales`. También creé todos los datos demo.

Valoración de Àngel:

Àngel asumió la parte visual con buen criterio. Amplió la ficha de contacto con la pestaña de pádel y actualizó el formulario de pistas con los campos nuevos. Añadió las vistas de gráfico, tabla dinámica y kanban, que son las que más impacto visual dan al proyecto. Mejoró el CSS con un tema verde deportivo y decoraciones en la lista. Llevó el peso de la documentación escrita.

8.3 Valoración del trabajo - Àngel Martínez

Autovaloración:

Me centré en las vistas XML e interfaz de usuario. Desarrollé la pestaña de pádel en la ficha de contacto con visibilidad condicional. Actualicé el formulario de pistas y añadí las vistas de análisis (gráfico, tabla dinámica y kanban) con tarjetas de colores distintos mediante clases CSS condicionales en QWeb. Mejoré el backend con CSS personalizado y me encargué de la documentación y la exposición oral.

Valoración de Rubén:

Rubén fue el motor técnico, organizó los módulos, configuró Git y montó el entorno Docker. Resolvió todos los errores técnicos difíciles de diagnosticar, como el problema del `addons_path` y los imports rotos. Implementó las partes más complejas: campos calculados encadenados, validaciones de solapamiento, el `@api.onchange` y la integración con el módulo de mensajería.

8.4 Metodología incremental

Se siguió una metodología incremental que permitió detectar errores progresivamente:

1. Análisis de necesidades del club.
2. Diseño conceptual del modelo de datos.
3. Creación de estructura del módulo.
4. Desarrollo de modelos Python.
5. Creación de vistas XML.
6. Integración con módulos estándar de Odoo.
7. Personalización gráfica (CSS).
8. Pruebas funcionales y de integración.
9. Implementación de mejoras (mail, onchange, graph/pivot).
10. Backup final de la base de datos.

9. Pruebas Realizadas

9.1 Pruebas funcionales

- Creación de pistas con todos los campos (nombre, código, tipo, precio, iluminación, superficie).
- Validación de unicidad de código de pista (constraint SQL).
- Validación de precio no negativo (@api.constrains).
- Creación de reservas y verificación de autocompletado de observaciones (@api.onchange).
- Validación de solapamiento de reservas (@api.constrains).
- Confirmación, realización y cancelación de reservas.
- Verificación de cambios de estado en el chatter.

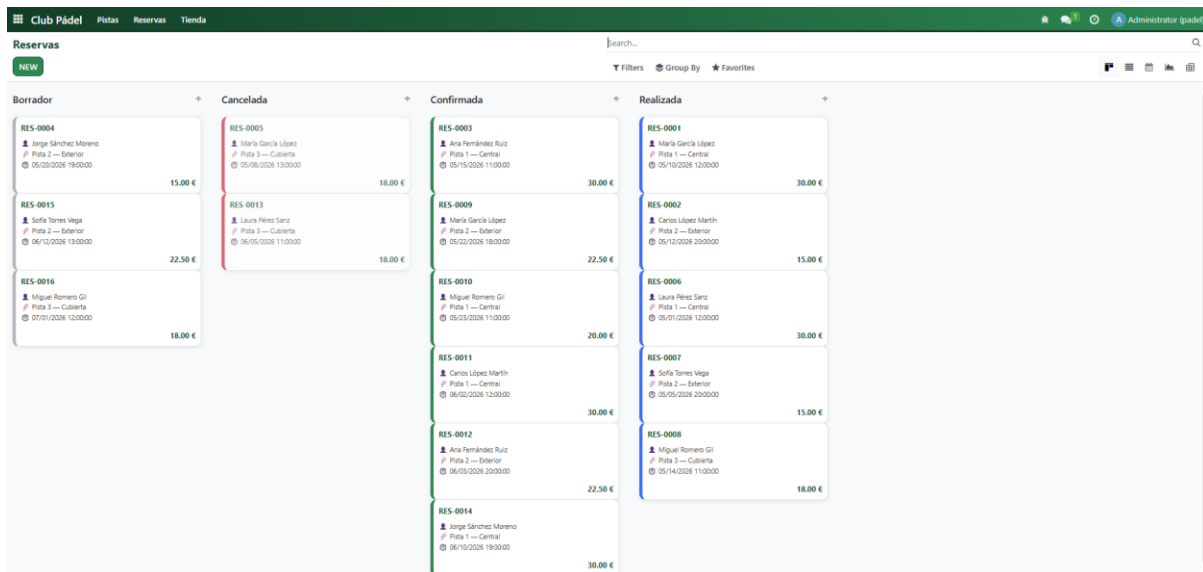
```
operador_odoo@ldapserver:~/custom_addons$ sudo -u operador_odoo /usr/bin/python3 /opt/odoo/odoo-bin \
--addons-path=/opt/odoo/addons,/opt/odoo/custom_addons \
-d bd_empresa1 -u club_padel --stop-after-init
operador_odoo@ldapserver:~/custom_addons$
operador_odoo@ldapserver:~/custom_addons$ sudo systemctl restart odoo16
[sudo] contraseña para operador_odoo:
operador_odoo@ldapserver:~/custom_addons$
```

9.2 Pruebas de integración entre módulos

- Creación de productos en el módulo tienda_padel.
- Asociación de productos a reservas mediante carritos (club_padel_tienda).
- Cálculo automático de totales en el carrito.
- Verificación de relaciones Many2one y One2many entre los tres módulos.
- Comprobación de vistas graph y pivot con datos reales de reservas.

9.3 Pruebas de consistencia y actualización

- Actualización de módulos con -u club_padel sin errores.
- Reinicio del servicio y comprobación de carga de assets CSS.
- Verificación de carga de CSS en modo ?debug=assets.



10. Rendimiento y Optimización

El sistema presenta baja carga inicial al tratarse de un entorno de tamaño medio. La arquitectura está preparada para el crecimiento gracias a la integración directa con el ORM eficiente de Odoo. No se realizaron optimizaciones avanzadas en esta fase, pero el sistema permite:

- Indexación futura de campos de búsqueda frecuente.
- Automatización de tareas programadas (cron jobs).
- Uso de workers múltiples en producción.

11. Escalabilidad del Proyecto

El sistema está preparado para las siguientes ampliaciones futuras:

- Implementar reservas online desde el portal web de Odoo.
- Automatizar facturación al confirmar una reserva.
- Integrar TPV (Terminal Punto de Venta) para la tienda.
- Crear dashboard con KPIs: ingresos mensuales, ocupación de pistas, ventas por producto, ranking de clientes.
- Control de torneos y ligas internas del club.

12. Plan de Mantenimiento

Se recomienda el siguiente plan de mantenimiento para garantizar la estabilidad del sistema:

- Backup semanal automático desde el gestor de bases de datos de Odoo formato .zip).
- Actualización de módulos de forma controlada con pruebas previas en entorno de desarrollo.
- Monitorización del servicio odoo16.service con systemctl status.
- Control de logs en /var/log/odoo/log.txt para detectar errores.
- Revisión trimestral de permisos de acceso.

```
operador_odoo@ldapserver:~$ ls /var/log/odoo/  
log.txt  
operador_odoo@ldapserver:~$
```

13. Valor Técnico del Proyecto

Este proyecto demuestra las siguientes competencias técnicas adquiridas:

Competencia	Aplicación en el proyecto
Análisis de necesidades reales	Digitalización de un club deportivo real
Implementación práctica de ERP	Odoo 16 Community en Ubuntu
Desarrollo de módulos propios	3 módulos con modelos, vistas y menús
Herencia de vistas y modelos	_inherit en res.partner
Decoradores ORM avanzados	@api.depends, @api.constrains, @api.onchange
Integración de assets CSS	web.assets_backend con CSS corporativo
Análisis de datos en Odoo	Vistas graph y pivot para KPIs
Integración de módulos Odoo	Módulo mail (chatter + tracking)
Diagnóstico técnico	Resolución de errores con IA y documentación
Gestión de entorno Linux	systemd, addons_path, PostgreSQL

14. Conclusión

La implantación realizada no solo ha digitalizado los procesos principales del club de pádel, sino que ha construido una arquitectura modular sólida basada en tres módulos propios: club_padel, tienda_padel y club_padel_tienda.

El sistema final permite gestionar reservas deportivas, perfiles de jugadores y ventas de productos en un entorno unificado. Las mejoras implementadas en la segunda fase — integración del chatter, decorador @api.onchange, vistas de análisis graph y pivot, y campos computed avanzados — elevan el nivel técnico del proyecto y demuestran una comprensión real de las posibilidades de personalización de Odoo.

El proceso de trabajo con herramientas de IA (ChatGPT en la fase inicial y Claude Code en la fase de mejoras) ha sido transparente y documentado: cada prompt, respuesta y aplicación queda recogido en este documento, cumpliendo con los requisitos del proyecto y demostrando un uso responsable y consciente de estas herramientas.

El resultado es una base digital profesional, escalable y lista para crecer junto con las necesidades del club.